Automata and Formal Languages

Notes made by Finley Cooper

12th October 2026

Contents

1 Introduction **3**

2 Regular Languages and Finite-State Automata **3**

2.1 Deterministic Finite-State Automata 3

2.2 Regular Languages 5

2.3 Non-determinism and ε-transitions 5

1 Introduction

In this course we will have $0 \in \mathbb{N}$.

Definition. (Alphabet) An *alphabet* Σ is a finite non-empty set with elements which are *symbols* or *letters*.

Definition. (String) A *string* or *word* of length m over Σ is $\alpha = x_1 x_2 \dots x_m$ with $x_i \in \Sigma$ or more formally $\alpha \in \Sigma^m$. We write $|\alpha| = m$ for the length of α.

Remark. We will denote the empty word as ε and denote

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n$$

and $\Sigma^\dagger = \Sigma^* \setminus \{\varepsilon\}$.

Definition. (Concatenation) If $\alpha = x_1 \cdots x_m \in \Sigma^m$ and $\beta = y_1 \cdots y_n \in \Sigma^n$ then $\alpha\beta$ is $x_1 \cdots x_m y_1 \cdots y_n \in \Sigma^{m+n}$.

For $0 \le i \le j \le m$, $x_{i+1} \cdots x_j$ is a substring of α of length $j - i$. So for $\alpha, \beta \in \Sigma^*$, β is a substring of α if and only if there exists $\gamma, \delta \in \Sigma^*$ with $\alpha = \gamma\beta\delta$.

Definition. (Homomorphism) For alphabets Σ, Π, a *homomorphism* $h : \Sigma^* \to \Pi^*$ satisfies $h(\alpha\beta) = h(\alpha)h(\beta)$ for all $\alpha, \beta \in \Sigma^*$.

Remark. This implies that $h(\varepsilon) = \varepsilon$.

Remark. Alternatively we can just consider a function $f : \Sigma \to \Pi$ which defines a unique homomorphism extending f by concatenation.

Definition. (Formal Langauge) A *formal language* L over Σ is *any* subset of Σ^*.

2 Regular Languages and Finite-State Automata

2.1 Deterministic Finite-State Automata

Remark. Suppose we want to describe a language L over Σ using a finite amount of information. We could extend Σ with other symbols to get an alphabet $\Pi \supset \Sigma$. and assign a word $w \in \Pi^*$ to L.

We can do this for countably many L. But we want w to *describe* L. We'll now see a countable class of languages where we can do this symbolically. We first give a computational method to determine these languages.

Definition. (Deterministic Finite-State Automata) A *Deterministic Finite-State Automata* (DFA) is a structure

$$D = (Q, \Sigma, \delta, q_0, F)$$

where,

(i) Q is a finite non-empty set of states;
(ii) Σ is an alphabet;
(iii) δ is a transition function from $Q \times \Sigma \to Q$;
(iv) Starting state $q_0 \in Q$;
(v) $F \subseteq Q$ is a set of acceptance states.

The idea for a DFA is that we input a word $w \in \Sigma^*$ we call $w = x_1 \cdots x_n$. The DFA starts in state q_0 and reads the first letter of the input string x_1. The DFA then transitions according to δ to $q_1 = \delta(q_0, x_1) \in Q$. We then repeat this process with x_2 and so on. Once the string is exhusted the string is accepted if $q_n \in F$ otherwise it is rejected. This defines a language.

We describe these with tables and diagrams.

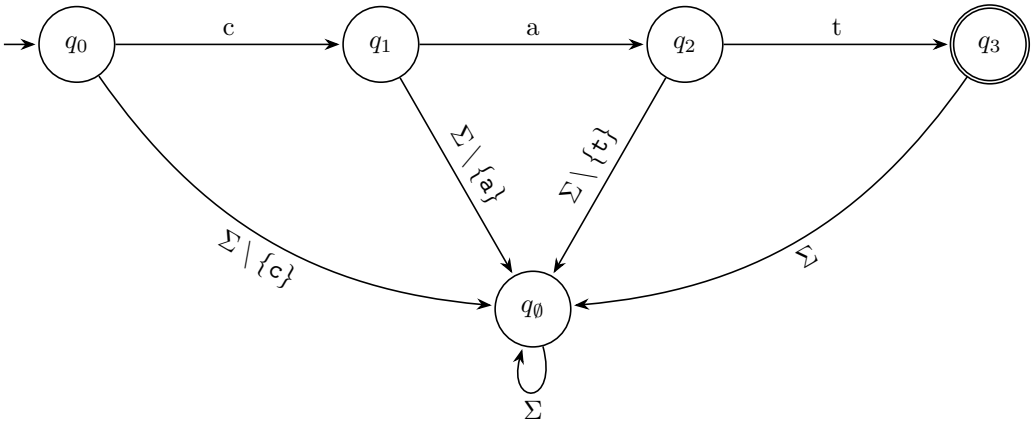
Definition. (Transition diagram) A *transition diagram* for a DFA, D, is a directed labelled graph with:

(i) The vertices being the states Q.
(ii) For each $(q, \sigma) \in Q \times \Sigma$, we draw a directed edge from q to $\delta(q, \sigma)$ and label this edge with σ. So each vertex has $|\Sigma|$ out-edges for every vertex.
(iii) Add an arrow from outside the diagram to q_0 labelled `START`.
(iv) Draw vertex as a circle or as a double circle if the state is an accept state.

If we have lots of edges from one state to another vertex, we can join them in a single edge with the label subset of Σ.

Consider the following diagram which shows a DFA which accepts the word 'cat' and nothing else.

$\Sigma = \{\texttt{a}, \texttt{b}, \ldots, \texttt{z}, \texttt{_}\}$ ($\texttt{_}$ denotes a space)



Definition. (Transition table) For a DFA we can also show the corresponding information in a table. The columns are symbols in Σ and the rows are states $q_0, q_1, \ldots$ and the (i, j) entry is $\delta(q_i, \sigma_j)$.

2.2 Regular Languages

Definition. (Extended transition function) The *extended transition function* of a DFA, D, is

$$\hat{\delta} : Q \times \Sigma^* \to Q$$

defined inductively via

$$\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, \sigma_1) = \delta(q, \sigma_1), \quad \delta(q, \sigma_1 \cdots \sigma_k) = \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k)$$

for all $q \in Q$ and $\sigma_1, \ldots, \sigma_k \in \Sigma$.

So for any word $w \in \Sigma^*$, $\hat{\delta}(q_0, w)$ is the end state with input w.

Lemma. For all $1 \leq \ell \leq k$, we have

$$\hat{\delta}(q, \sigma_1 \ldots \sigma_k) = \hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_k).$$

Proof. Induct on k. We take the base case where $k = \ell$ which follows from the definition. Otherwise,

$$\begin{aligned} \text{LHS} &= \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k) \\ &= \delta(\hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_{k-1}), \sigma_k) = \text{RHS} \quad \square \end{aligned}$$

Definition. (Regular Language) Let D be a DFA. The langauge $\mathcal{L}(D)$ of D is the set of words accepted by D namely

$$\mathcal{L}(D) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \right\} \subseteq \Sigma^*$$

A language L is *regular* if there exists a DFA D such that $L = \mathcal{L}(D)$.

2.3 Non-determinism and ε-transitions

We now examine a seemingly more general construction where we can choose which state to move to (or equivalently move to many states simultaneously).

Definition. (Non-deterministic Finite-State Automaton) A *non-deterministic finite-state automaton* (NFA) is a structure

$$N = (Q, \Sigma, \delta, q_0, F)$$

just as a for a DFA, however δ is now a map $Q \times \Sigma \to \mathcal{P}(Q)$, to contain the idea of moving

to many states "at once".

An NFA as input $w = \sigma_1 \dots \sigma_k \in \Sigma^*$ while in state q_0, reads σ_1 then "moves to" all of the states in $\delta(q_0, \sigma_1)$. Then for all states $p \in \delta(q_0, \sigma_1)$, read σ_2 and move to $\sigma(p, \sigma_2)$. Continue to end of w to get to all end states $E_w \subseteq Q$. If $E_w \cap F \neq \emptyset$, then w is accepted, otherwise w is rejected.

Remark. Note that since $\emptyset \in \mathcal{P}(Q)$ we can have $\delta(p, \sigma) = \emptyset$. But what if we're currently in states $S \subseteq Q$ and to read σ_i only to find that $\bigcup_{p \in S} \delta(p, \sigma_i) = \emptyset$. Then we are in "no state" and we are stuck in NFA limbo until the end of the word, where the word is automatically rejected.

Given an NFA N, we can define the transition diagram and the transition table in the same way as for a DFA although the table now contains subsets of Σ and the diagram has multiple directed edges for a single letter in Σ.

Definition. (Extended transition function) For an NFA the *extended transition function* $\hat{\delta}$ of N is $\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$ defined by $\hat{\delta}(q, \varepsilon) = \{q\}$ for $q \in Q$, $\hat{\delta}(q, \sigma) = \delta(q, \sigma)$ for $q \in Q$ and $\sigma \in \Sigma$ and

$$\hat{\delta}(q, \sigma_1 \dots \sigma_k) = \bigcup_{p \in \hat{\delta}(q, \sigma_1 \dots \sigma_{k-1})} \delta(p, \sigma_k).$$

So for $w \in \Sigma^*$ we have that $\hat{\delta}(q_0, w)$ is the set of end states E_w.